Multi2Sim Kepler: A detailed architectural GPU simulator

Xun Gong, Rafael Ubal, David Kaeli
Dept. of Electrical and Computer Engineering
Northeastern University
Boston, MA 02115
gong.xun@husky.neu.edu, ubal@ece.neu.edu, kaeli@ece.neu.edu

***Abstract*—Presilicon simulation is one of the key toolsets for computer architects to evaluate and optimize their future designs. As Graphics Processing Units (GPUs) have become the platform of choice in many computing communities due to their impressive processing capabilities, computer architecture researchers need a simulation framework that allows them to quantitatively consider design tradeoffs.**

In this paper, we present the Multi2Sim Kepler simulator framework, a new detailed GPU microarchitecture performance simulator that supports NVIDIA's Kepler shader assembly (SASS) code execution. The toolset provides a disassembler, a functional simulator and a detailed cycle-based simulator. We provide insight into the architecture of the NVIDIA Kepler GPU, describing the details of the streaming multiprocessor, front-end and instruction pipelines. We compare the performance of this new simulator against an NVIDIA K20X, a high-end Kepler device. We also evaluate the performance of NVIDIA's CUDA benchmark suite on our GPU performance simulator.

I. Introduction

GPUs have become the accelerator of choice in a range of application areas, including High Performance Computing (HPC) and full-feature smartphones. An increasing number of supercomputers in the TOP500 have been equipped with GPUs as accelerators. Modern GPUs, such as NVIDIA's Fermi, Kepler and Maxwell devices have hundreds to thousands of processing cores. NVIDIA and Kronos have introduced high-level programming models, CUDA [1] and OpenCL [2], that have enabled a large number of applications to be ported to a GPU.

Given the growth in GPU computing, the academic computer architecture research community requires accurate simulation tools. Simulation frameworks provide researchers the ability to quantitatively evaluate architectural designs and evaluate various tradeoffs. A number of GPU simulators have been developed that support architectural exploration. However, current GPU simulators that support CUDA program execution on an NVIDIA-like GPU only support execution of intermediate language (i.e., PTX). The research community lacks an open-source simulation framework which runs cycle-accurate detailed architectural simulation at the GPU SASS level (i.e., the executable binary level).

In this paper we present Multi2Sim Kepler, a GPU simulation framework for evaluating GPU computing based on the NVIDIA Kepler architecture. This paper makes the following contributions:

- We provide a detailed description of the Streaming Multiprocessor (SM) architecture design based on the NVIDIA Kepler GPU.
- We discuss the benefits of supporting native SASS code running on the simulator and evaluate the differences between simulating at a higher level intermediate language (PTX) versus working at a SASS binary level.
- We demonstrate the utility of Multi2Sim Kepler, modeling an SM with shared execution resources, showing how resource partitioning can help reduce contention in the SM.
- We further show the benefits of this infrastructure by producing an analysis of application characteristics, as well perform microarchitectural studies.

This paper is organized as follows. In Section II we describe our general development methodology for Multi2Sim. In Section III we describe the current CUDA support provided for in the simulator. We present a discussion on microarchitecture design choices, as well as NVIDIA's SASS in Section IV. In Section V we describe our baseline GPU model, present various performance studies using our simulator, and discuss simulation results. Section VI reviews related work. We conclude the paper in Section VII, and suggest directions for future work.

II. Simulation Methodology

The development of our Kepler model in Multi2Sim follows three phases of development, as presented in Figure 1, with the phases flowing from left to right. To deliver a cycle-accurate simulator, development involves the design and implementation of three independent software modules: i.) a disassembler, ii.) a functional simulator and iii.) a detailed simulator.

A dissembler can be used standalone to disassemble or can drive later simulation stages (i.e., emulation or timing simulation). In the first case, the disassembler reads directly from a program binary generated by a compiler, such as CUDA application binary, and dumps a text-based output of all fragments of the ISA code found in the file. In the second case, the disassembler reads from a binary buffer in memory and outputs machine instructions one by one in the form of

978-1-5386-3890-3/17/$31.00 ©2017 IEEE

custom data structures that break down each instruction into its various fields.

The purpose of the functional simulator, also called the emulator, is to reproduce the original behavior of the CUDA program, providing the illusion that it is running natively on an NVIDIA GPU. Functional simulation is especially handy for debugging programs, and gathering statistics about the general behavior of a workload.

The detailed simulator, interchangeably referred to as the timing or architectural simulator, is the software component that models hardware structures in the Kepler GPU, and keeps track of execution time. The modeled hardware includes Streaming Multiprocessors (SMs), functional units, pipeline stages, pipe registers, instruction queues, cache memories, and other features.

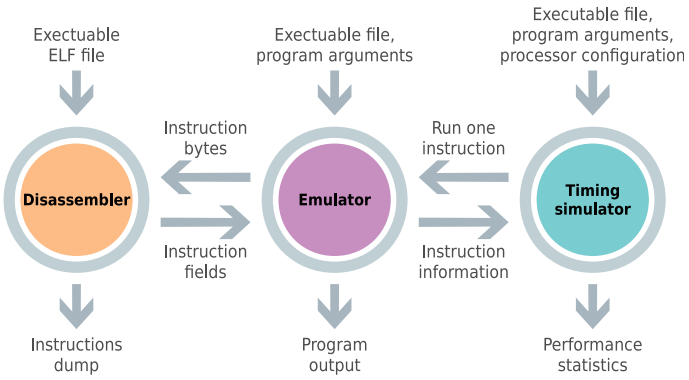


Fig. 1. Kepler's simulation paradigm.

III. CUDA support on Multi2Sim Kepler

The Multi2Sim Kepler model is a new GPU architecture extension to our existing Multi2Sim framework. Multi2Sim is a free, open-source, cycle-accurate heterogeneous simulation framework that can model superscalar, multithreaded, and multicore x86 CPUs, ARM CPUs, MIPS CPUs, AMD GPUs, and now NVIDIA GPUs.

The current version of our Kepler functional simulator supports the execution of a number of different benchmarks without any porting effort by the user. It also supports the NVIDIA CUDA SDK benchmarks, as well as custom CUDA benchmarks we have developed. The architectural simulator models a detailed NVIDIA Kepler GPU, complete with an associated memory hierarchy, interconnection network and additional components.

Multi2Sim supports truly heterogeneous execution. Every simulated program begins execution on the CPU. Multi2Sim supports unmodified NVIDIA CUDA execution on the Kepler GPU simulator. When CUDA programs are executed, the host (i.e., the CPU) portion of the program is run using the CPU simulator. When CUDA API calls are encountered, they are intercepted and transfer execution to the Kepler simulation model.

A. CUDA Programming Model

CUDA is a parallel computing platform and application programming interface (API) model created by NVIDIA. CUDA enables software developers to access the GPU's virtual instruction set and parallel computational elements directly for compute kernel execution. With millions of CUDA-enabled GPUs in use today, software developers, scientists, and researchers are finding new uses for GPU computing with CUDA [3].

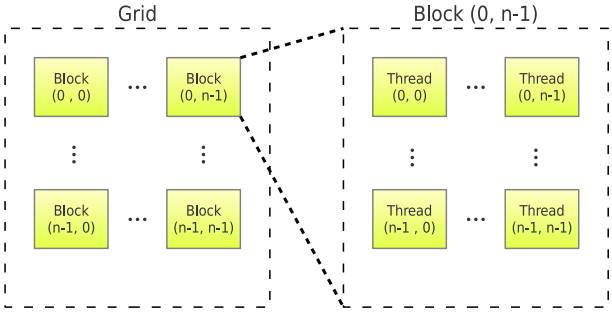


Fig. 2. The CUDA programming model.

Figure 2 provides an overview of the execution element hierarchy defined in CUDA. CUDA C extends C by allowing the programmer to define C functions, called kernels. When a kernel is called, N copies of the kernel are executed in parallel by N different CUDA threads. An instance of a CUDA kernel is called a thread, which can access its own pool of local memory. Within a grid, threads are grouped into blocks with two basic properties: i.) within a single block, threads can perform efficient synchronization operations, and ii.) threads within the same block can share data through a low-latency shared memory. All thread blocks form a grid and share a common global memory.

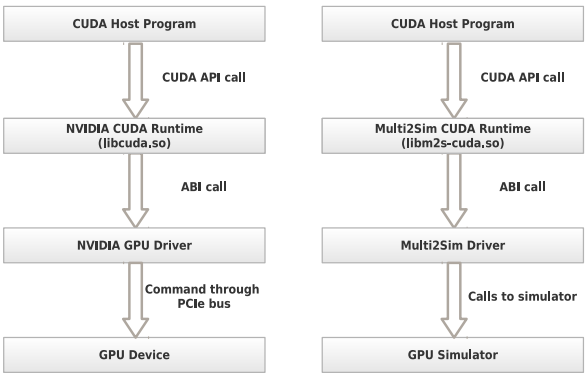


Fig. 3. Comparison of native and simulated Kepler execution environments.

B. CUDA Simulation

The call stack of a CUDA program running on the Multi2Sim simulator differs from the native call stack, starting at the CUDA library call, as shown in Figure 3. We implemented a series of CUDA runtime [4] and driver [5] APIs,

as well as a GPU device driver for the simulator. When a CUDA API function call is issued, our implementation of the CUDA runtime (libm2s-cuda.so) handles the call. This call is intercepted by the CPU simulation module, which transfers control to the GPU module as soon as the guest application launches the device kernel. This infrastructure allows unmodified x86 binaries (i.e., precompiled CUDA host programs) to run on Multi2Sim with total binary compatibility with the native environment.

IV. Architectural Simulation Implementation

Next, we provide an overview of our simulator infrastructure and describe the architecture of a generic NVIDIA Kepler GPU device [6]. We focus our discussion on hardware components devoted to general purpose computing of CUDA kernels (versus the graphics pipeline). We provide insights into the GPU architectural details of the Kepler architecture [7], including the frontend, streaming multiprocessor design, and instruction pipeline. Note that many of these element remain undocumented in publicly available information from NVIDIA.

A. The Kepler GPU Architecture

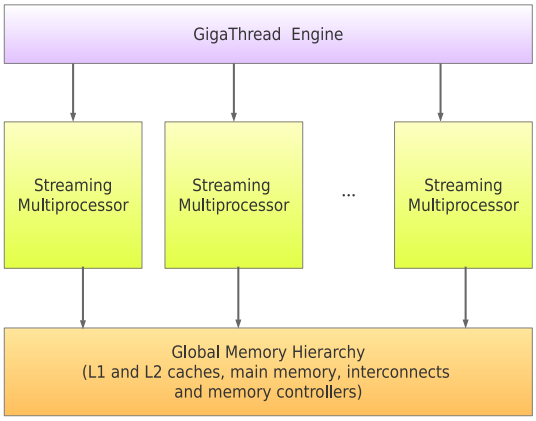


Fig. 4. GPU architecture.

Figure 4 presents an overview of the Kepler GPU architecture supported in Multi2Sim. The GPU organization includes a giga-thread engine, a collection of SMs and a memory hierarchy. The giga-thread engine processes the grid and maps waiting thread blocks onto the available SMs. Once a thread block is assigned to an SM, it remains in the SM until its execution completes. Resources, such as registers and shared memory, are not freed until the thread block finishes execution. Multiple thread blocks can be assigned to a single SM if there are available resources.

A detailed description of an SM is shown in Figure 5. An SM consists of a frontend, a register file and a number of execution units. CUDA defines the concept of a warp as a group of 32 threads executing in Single-Instruction Multiple Data (SIMD) fashion. Each instruction is executed concurrently by every thread comprising a warp, although each thread has its own data for computation. This model simplifies

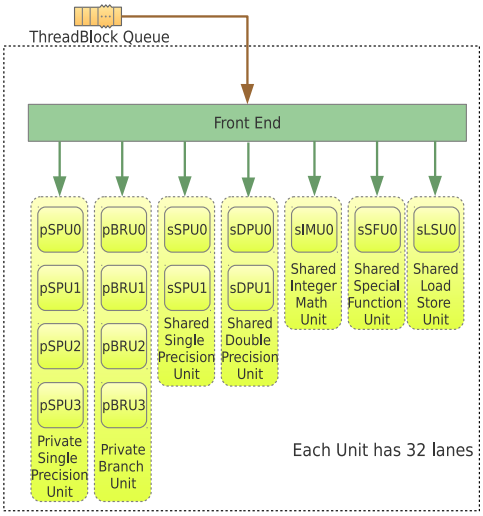
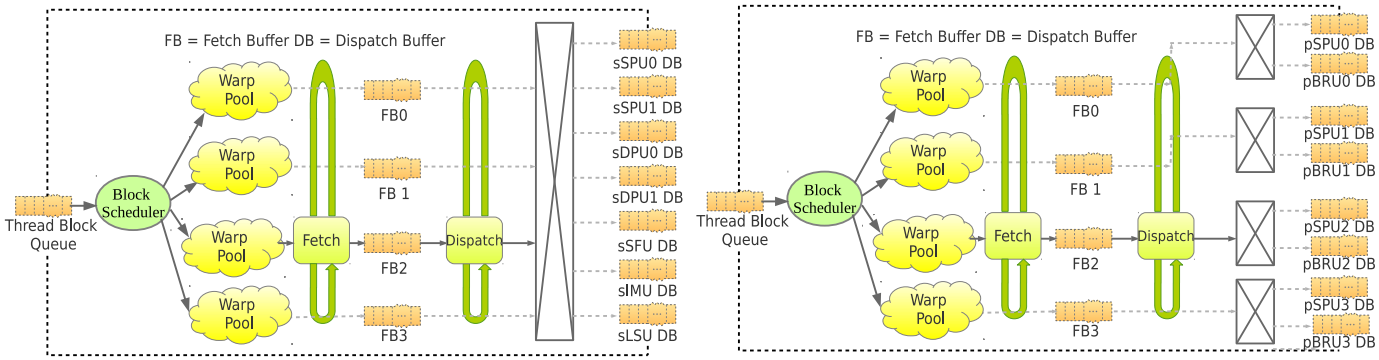


Fig. 5. The SM architecture.

instruction fetch hardware by implementing a common front-end for a whole warp. On every cycle, the SM's front-end fetches instructions from instruction memory for the different warps and sends them to the appropriate execution unit. In the Kepler microarchitecture, each SM front-end features four warp schedulers and eight instruction dispatch units, allowing four warps to be issued and executed concurrently.

The execution units present in an SM are the branch unit (BRU), the load store unit (LSU), the single precision unit (SPU), the double precision unit (DPU), the special function unit (SFU) and the integer math unit (IMU). Each unit consists of 32 lanes (32 CUDA cores in NVIDIA terminology), and each lane is devoted to the execution of one thread's arithmetic operation. This means that each unit can allow all 32 threads in a warp to execute the same instruction simultaneously. In Kepler, there are only two kinds of execution units. The first kind is a private unit, which is owned by each warp scheduler and can only execute SIMD instructions from the scheduler it belongs to (i.e., a private single precision unit (pSPU) and a private branch unit (pBRU)). The other kind of unit is a shared unit, which is shared among the 4 warp schedulers (i.e., a shared single precision unit (sSPU) and a shared integer math unit (sIMU)). A shared unit can execute SIMD instructions from either one of the four warp schedulers.

Figure 6 shows the architectural details of the SM front-end. This includes a set of warp pools, a fetch stage, a set of fetch buffers and a dispatch stage. The number of warp pools matches exactly the number of warp schedulers in an SM. When blocks are initially mapped to an SM, their associated warps are equally distributed to the available warp pools. In the fetch stage, the warps are selected from the pool, instructions are read from instruction memory using the warp's current program counter (PC), and these instructions are placed in the associated fetch buffer. In the next cycle, the fetch stage operates on the next warp in the warp pool, following a round-robin policy. In our model, we fetch a



(a) The front-end pipeline for shared units.

(b) The front-end pipleline for private units.

Fig. 6. A block diagram of the front-end pipeline.

maximum of 2 instructions per warp, per cycle, depending on resource availability. At the beginning of the dispatch stage, the recently fetched instructions are decoded and stored in their corresponding entries in the buffer. The warp scheduler can pick a warp to issue to the available warps. Before issuing the ready instructions to the execution units, the scoreboard performs dependency checks and generates a ready bit if the instruction passes the check. If the instruction is ready to dispatch, and there are still available resources, it is sent to the corresponding unit to execute.

B. Kepler Instruction Set Architecture (ISA)

NVIDIA has developed several major architectures such as Fermi (SM 2.x) [8], Kepler (SM 3.x) [6], and Maxwell (SM 5.x) [9]. Within those families, new instructions have been added as NVIDIA updated their accelerators. For example, Kepler added a 64-bit funnel shift instruction that concatenates two 32-bit values together, shifts the resulting 64-bit value left or right, and then returns the most significant or least significant 32 bits.

```
/*00e0*/        PBK 0x190;              /*0130*/        FMUL R5, R3, R0;
/*00e8*/        ISETP.LT.AND P0, PT,    /*0138*/        SHL R3, R18, 0x2;
                R18, R17, PT;           /*0148*/        IADD R4, R2, R3;
/*00f0*/        PSETP.AND.AND P0, PT,   /*0150*/        MOV R3, R4;
                !P0, PT, PT;            /*0158*/        LD R3, [R3];
/*00f8*/    @P0 BRK;                    /*0160*/        FMUL R5, R3, R5;
/*0108*/        BRA 0x110;              /*0168*/        MOV R3, R4;
/*0110*/        SHL R3, R18, 0x2;       /*0170*/        ST [R3], R5;
/*0118*/        IADD R3, R16, R3;       /*0178*/        IADD R18, R18, R19;
/*0120*/        MOV R3, R3;             /*0188*/        BRA 0xe8;
/*0128*/        LD R3, [R3];
```

Fig. 7. Sample SASS code.

When the GPU functional simulator receives the CUDA kernel to execute, an emulation loop starts executing Kepler instructions. The basic format of the NVIDIA Kepler ISA can be observed in the sample code in Figure 7. This block of native NVIDIA SASS code represents a *for-loop* in a CUDA kernel. It starts with a PBK instruction which specifies the target address to resume execution after the end of the loop, working with the BRK instruction to control the loop. The ISETP and PSETP instructions are used to calculate the predicate register value, deciding whether we are going to jump out from the loop.

1) Comparison between SASS and PTX : PTX [10] is a low-level parallel-thread-execution virtual machine and instruction set defined by NVIDIA. PTX is a high-level intermediate language as compared to native SASS. There are two compilation stages required before a kernel written in CUDA can be executed on the GPU. The first stage compiles the CUDA code into the PTX virtual GPU ISA. The second stage compiles PTX into the actual SASS. The SASS code changes for each different generation of NVIDIA GPU, while PTX code is architecture independent.

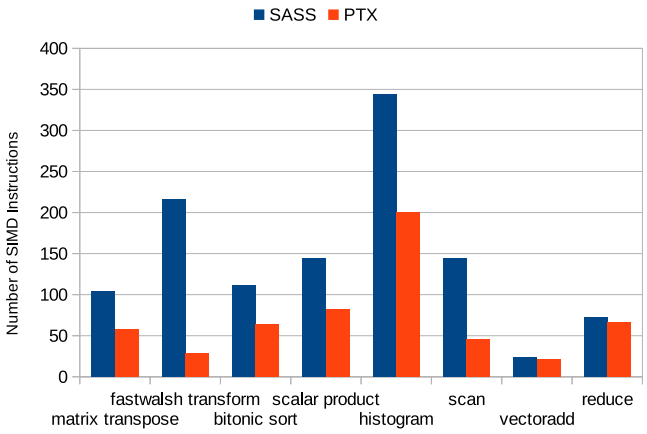


Fig. 8. The number of SIMD instructions in SASS and PTX.

The number of SIMD instructions from our set of benchmarks is shown in Figure 8. In all the kernels we tested, the number of SIMD instructions in SASS is more than the number in PTX. Additionally, in some kernels, the number of SIMD instructions executed in SASS is 8 times larger than in PTX (e.g., the Fast Walsh Transform). We analyzed the kernel and found that there is a reciprocal (RCP) operation in the code. When the real hardware executes the RCP operation, it requires many extra instructions to perform overflow protection, which makes the number of SIMD instructions in SASS much larger.

From our discussion above, we can observe a couple of important differences between working with SASS versus PTX. PTX instructions are not mapped to SASS 1 for 1. In Kepler SASS, there are a limited number of general purpose registers and a large number of instructions, so there are more restrictions when the scheduler decides whether the instruction can be dispatched to the execution units. In general, there are many ISA-specific issues when we run SASS that need not be considered when working with an intermediate language. Thus, running SASS simulation is much closer to the actual GPU execution in those aspects.

2) CUDA Kernel Execution : When a CUDA kernel is launched by a host program, the kernel configuration is provided to the GPU. Thread blocks are then created and successively assigned to SMs when there are execution resources available. The number of thread blocks that can be assigned to a single SM is determined by following hardware constraints: i) the maximum number of thread blocks supported per SM, ii) the maximum number of warps allowed per SM, iii) the number of registers on an SM, and iv) the amount of shared memory on an SM. Maximizing the number of assigned thread blocks per SM can have a tremendous impact on overall performance, which can be easily evaluated in our simulator.

Each block assigned to an SM is partitioned into warps, which are then placed into 4 warp pools equally. The warp scheduler selects warps from the warp pools for execution, based on a warp scheduling algorithm. The benefits of different warp scheduling algorithms can also be evaluated in our simulator.

3) Control flow and branch divergence: Branch divergence leads to performance degradation in the SIMD execution model when threads within a single warp take different paths. The Kepler ISA provides each warp with an active mask in order to resolve branch divergence present during SIMD instruction execution. The active mask is a 32-bit vector, where each bit indicates whether the corresponding thread is active (or not) in the warp. If a thread is labeled as inactive, the result of any operation performed in the associated lane is ignored, preventing the thread from changing the kernel state.

An active mask stack is used to support control flow instructions by pushing and popping active masks. When branch divergence happens, active masks, representing different paths, are pushed onto the stack, and the threads in the active mask, at the top of the stack, keep executing. The control paths taken by the threads in a warp are traversed one at a time until there are no more paths. Different execution paths are executed serially. All threads will re-converge when all the paths are finished, with the active masks representing the different paths being popped from the stack.

C. The instruction pipeline

In an SM, different kinds of instructions are sent to the appropriate execution unit by the dispatcher. Figure 9(a) presents a block diagram of the math instruction pipeline. Within each pipeline, decisions about latencies, scheduling policies, and buffer sizes must be made. All of these factors have performance implications and provide another opportunity for architecture researchers to experiment with design tradeoffs using our simulator.

The Kepler SM features an SPU (both shared and private), DPU, IMU and SFU for arithmetic operations. Each unit has 32 lanes. Each SPU has fully pipelined floating-point and integer arithmetic logic units. The Kepler SM also uses the SFU for fast approximate transcendental operations, the IMU for complex integer operations (e.g., integer scalar adds (ISCADD) and bit field extractions (BFE)) and the DPU for double precision operations.

After the fetch and dispatch stages, decoded instructions are placed into the dispatch buffer. The read stage consumes the instruction and reads the source operands from the register file for each thread in the warp. The execute stage issues an instance of the instruction to each lane in the unit on every cycle. The number of lanes in a unit in the Kepler matches the number of threads in a warp. The result of the computation is written back to the destination register in the write stage.

Similar pipelines are provided for the BRU and the LSU, as shown in Figures 9(b) and 9(c), respectively. In the Kepler SM, the LSU handles all the load and store operations from the memory hierarchy, including global memory, shared memory and private memory operations. This design is different from AMD's, where shared memory operations and global memory operations are handled by different units.

D. The Memory hierarchy

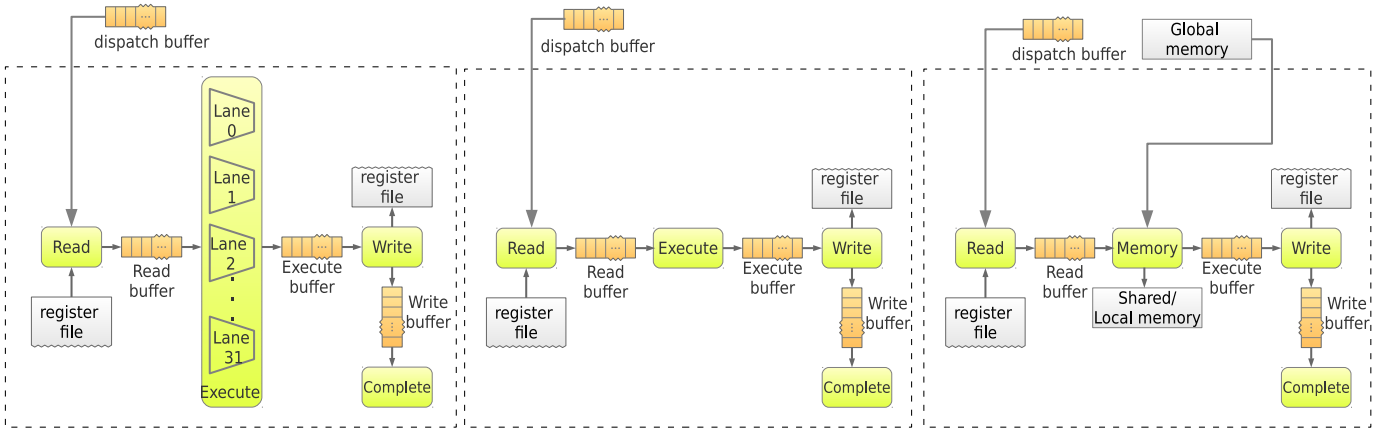
The GPU memory subsystem contains different regions for data storage (global, shared, local). In our Kepler simulator, we use the Mult2Sim's existing memory model, which is highly configurable. The model includes customizable settings for the number of cache levels, memory capacities, block sizes, number of banks and ports.

In our model, each thread has non-contended access to the register file. Multiple warps referencing the many units can access the register file, simultaneously, in a given cycle. A separate shared memory module is present in each SM. GPU global memory is accessible by all SMs. In Multi2Sim, the global memory hierarchy has a configurable number of cache levels and interconnects. Each cache in the global memory hierarchy is connected to the lower-level cache (or global memory) using an interconnection network. Interconnects are organized as point-to-point connections using a switch, which contain two disjoint inner subnetworks, each devoted to package transfers in opposite directions.

V. Experimental Evaluation

To demonstrate the utility of Multi2Sim, we present a set of experiments for validating and demonstrating both functional and architectural simulation features discussed in Section IV. All experiments are based on a baseline GPU model very similar to the NVIDIA Telsa K20X [11], whose hardware specifications are summarized in Table 1.

We elected to utilize a representative subset of the NVIDIA CUDA SDK benchmarks. The benchmarks included provide



(a) Block diagram of the execution pipeline for SPUs, DPUs, IMUs, SFUs.

(b) Block diagram of BRU pipeline.

(c) Block diagram of LSU pipeline.

Fig. 9. Pipeline of different units.

TABLE I
BASELINE GPU CONFIGURATIONS

device configuration		
compute resources	number of SM	14
	Warp size	32
	Number of SPUs / SM	6
	Number of BRUs / SM	4
	Number of DPUs / SM	2
	Number of IMUs / SM	1
	Number of SFUs / SM	1
	Number of LSUs / SM	1
	Number of lanes / Unit	32
Register file & Shared memory	Number of registers / SM	65536
	Shared memory / SM (KB)	16
L1 Cache	Associativity	4
	Block size	128B
	Total L1 cache size	16KB
L2 Cache	Associativity	16
	Block size	128B
	Total L2 cache size	1536KB
Others	Max warps / SM	64
	Max thread blocks / SM	16
	Max threads / SM	2048

TABLE II
LIST OF CUDA BENCHMARKS USED FOR EVALUATION

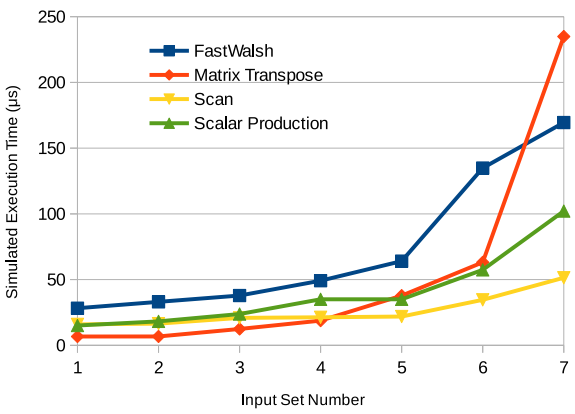
Benchmark	Input feature	Configuration
Bitonic Sort (BS)	Vector size	256 - 16k
Fast Walsh Transform (FW)	Array size	1k - 512k
Histogram (HM)	Array size	8k - 1M
Inline Math (IM)	Grid size	32k - 1280k
Matrix Multiplication (MM)	Matrix sizes	64 x 64 - 512 x 512
Matrix Transpose (MT)	Matrix size	128 x 128 - 1k x 1k
Scalar Production (SP)	Vector size	32k - 1M
Scan (SC)	Array size	4k - 1M
Vector Addition (VA)	Array size	64k - 1M

us with a rich set of compute-intensive and memory-intensive benchmarks, as well as a mix of both, representing a wide range of application behaviors. The benchmarks discussed in this paper are listed in Table 2, along with some of their application properties. We include a description of the input datasets and a summary of the programs.

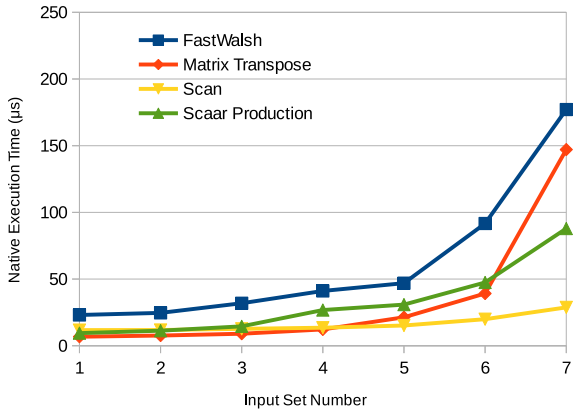
A. Baseline Model Validation

We present our simulator validation process in a number of steps. First, we start by validating our runtime library and device driver to make sure the device is properly initialized. Second, for the most important elements in our simulated architecture, we follow two different validation strategies for considering the correctness of both the functional and the architectural simulation models. For the functional simulator, we validate our instruction decoder by comparing the disassembled SASS code with the output generated by the NVIDIA disassembler. We also validated the correctness of each benchmark by comparing the simulated application output with the output of the application run directly on the K20X. All simulations generate correct results for all the benchmarks and for different input sizes.

In terms of the precision of our architectural model, we compare our simulation results against the performance of the NVIDIA Tesla K20X hardware for the benchmarks and input sizes shown in Table II. The native execution time is calculated as the average time of 1000 kernel executions for each benchmark while varying the input size. We measured the hardware execution time using the NVIDIA profiler, nvprof. Since our architectural simulator is a cycle accurate model and native execution performance on the GPU is measured



(a) Simulated execution time reported by Kepler Simulator.



(b) Native execution time on NVIDIA Tesla K20X.

Fig. 10. Validation for the architectural simulation, comparing trends between simulated and native execution times.

by execution time, it is a bit challenging to compare the two results directly. In order to measure our architectural model, we use the K20X shader base clock frequency of 732 MHz documented by NVIDIA to turn Multi2Sim cycles into execution time.

The differences in simulated and native performance are plotted in Figures 10(a) and 10(b) (only four are shown for clarity, the other workloads followed a similar trend). In all of the benchmarks, the simulated execution time and the native GPU execution time exhibit the same trend, increasing the execution time that follows the growth in input size. In other words, a change in the problem size for a benchmark has the same relative performance impact for both native and simulated execution.

Figure 11(a) plots the simulated and native execution time. In Figure 11(b), we show a scatter plot of the normalized simulation execution time obtained for our simulator versus the normalized execution time, measured using NVIDIA K20X GPU hardware. The the points that lie closest to the trend-line indicate that the simulated and native execution times track each other. For some of the benchmarks, execution times vary significantly. One outlier is the HM benchmark, as seen in Figure 11. The execution time produced by Multi2Sim is more than 2X the execution time on the K20X hardware. This outlier is due HM's heavy use of ST.WT and LD.CV instructions. These instructions change the cache policy, and are not presently modeled faithfully in Multi2Sim's memory model. Besides this one source of inaccuracy (that is presently being updated), we have also identified the following sources of imprecision in our model:

Cache Parameters & Interconnects. Some sources of simulation inaccuracy can be attributed to cache parameters, since the latencies of the different levels of the cache hierarchy have not been made public by NVIDIA. The specification of the interconnect network between the L1 and L2 caches has also been approximated due to a lack of public information. We continue to improve upon this model through reverse engineering.

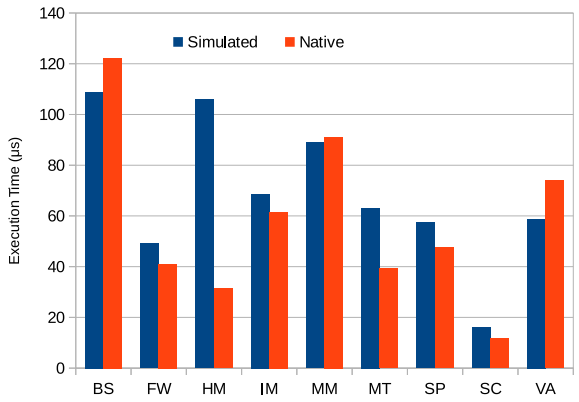
Execution resource partitioning. As we discussed in Section 3, in a Kepler SM, there are shared execution resources (e.g., sSPUs, sIMU and sDPUs) which are shared across all four warp schedulers. But there is no documentation detailing how they share resources in each cycle. We provide a further discussion on this point in the next subsection.

However, we still see a strong correlation between native execution and the associated simulator results for all benchmarks. In general, the benchmarks that perform well on the K20X hardware perform well in our simulator. The benchmarks that exhibit poor performance also have poor performance in our simulator.

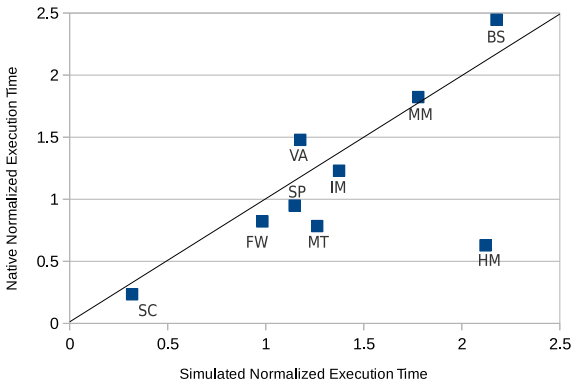
B. Effect of Execution Resource Partitioning

On the GPU, there are several execution units which are shared across all 4 warp schedulers in each SM. These include: DPUs, SFU (each unit has 32 lanes which can execute 32 threads simultaneously). In this section, we explore the impact that different partitioning strategies have on performance. We study and compare two partitioning algorithms, one that performs even partitioning (EP), allowing each warp scheduler to have a quarter of each shared unit (8 lanes). It will take 4 consecutive cycles for a warp to execute all 32 threads if its SIMD instructions are dispatched only to its own subset of the shared units. The second participating scheme is static partitioning (SP). With this scheme we dispatch the SIMD instructions to the appropriate unit from the warp scheduler whose index is the lowest among all qualified schedulers. In this case, it takes 1 cycle for a warp to execute all 32 threads when the SIMD instruction is sent to one of the shared units.

Figure 12 compares the performance between our two partitioning algorithms, EP and SP. All of the benchmarks experience a speedup when choosing EP over SP. Scan has the least speedup across all of the benchmarks, which is nearly 40%. The other benchmarks are even higher. Using our built-in performance metrics in Multi2Sim, we find that EP provides higher utilization of execution resources than SP.



(a) Comparision between simulated execution time and native execution time.



(b) Normalized execution time.

Fig. 11. Validation for the architectural simulation, comparing simulated and native execution times.

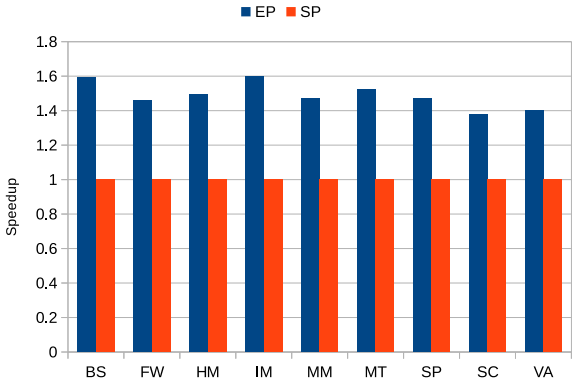


Fig. 12. Performance comparison between EP and SP.

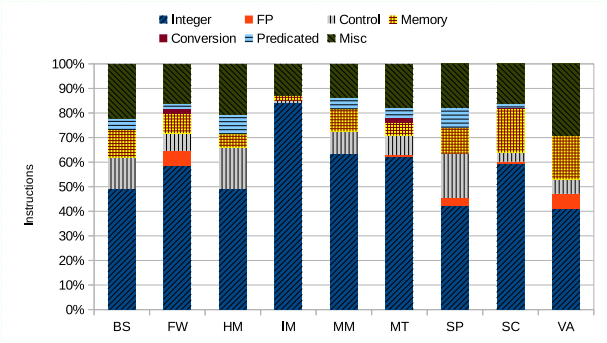


Fig. 13. Instruction classification.

When choosing SP, the warps dispatched by warp scheduler 0 are always faster than the warps dispatched by the other schedulers. When all warps dispatched by warp scheduler 0 are finished, the warps from the other schedulers are still running, without using the private units belonging to warp scheduler 0. Furthermore, when there are only warps from the last warp scheduler left, all the private units belonging to the other three warp schedulers are freed, which results in very low utilization of execution resources.

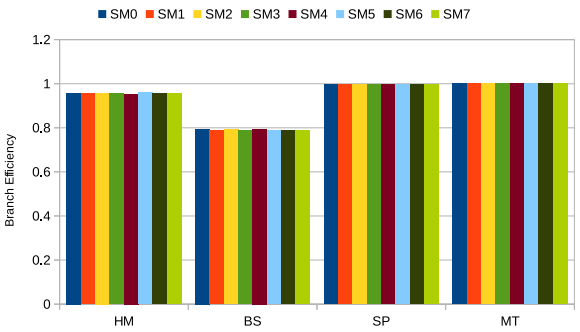
C. Benchmark Characterization

To further illustrate the power of our infrastructure, we present a characterization of the CUDA benchmarks used in our study. We characterize programs based on instruction mix, branch efficiency and control flow efficiency. These statistics are dynamic in nature, and are reported by our Kepler simulator as part of its simulation reports.

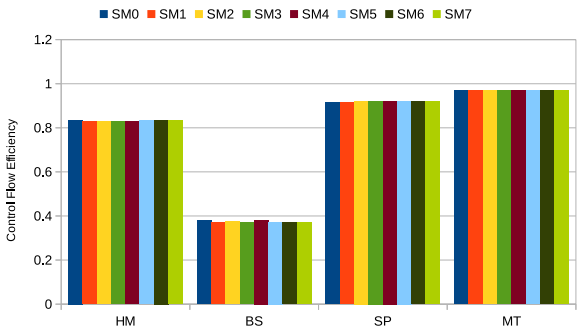
Figure 13 shows Kepler SASS instruction mixes for each benchmark. The instruction categories include: integer, floating point (FP), control flow instructions (branch, function call, jump and etc.), memory, data conversion (different data type conversions, such as integer to floating point), predicate (predicate register operations) and miscellaneous instructions (e.g., NOP, BAR, etc.).

Figure 14(a) shows the branch efficiency (the ratio of uniform control flow decisions over all executed branch instructions) per-SM (only 8 SMs are shown due to space limitation). This metric can be used to gauge the overall divergence rate. Different SMs have a similar branch efficiency in all of the benchmark tested because they did similar amounts of work. Figure 13 shows that MT, BS, HM and SP contain from 8 % to 18 % control flow instructions. However, control flow intensity does not necessarily translate into high branch divergence. It depends more on the sensitivity of individual thread execution in a warp. Although SP has a highest percentage of control flow instructions across the four benchmarks, only 0.1 % of them translate to a branch divergence (shown in Figure 14(a)). In contrast, BS has 12 % control flow instructions, but 21% of them result in divergent execution.

Branch efficiency is a interesting metric which can provide us with the divergence ratio of the branch instructions in a benchmark. However, this metric does not have any knowledge of the extent of the code length generated by the divergence. This information is insufficient to debug the impact of a divergence with respect to kernel execution performance. For



(a) Branch efficiency for different SMs.



(b) Control efficiency for different SMs.

Fig. 14. Branch efficiency and control flow efficiency.

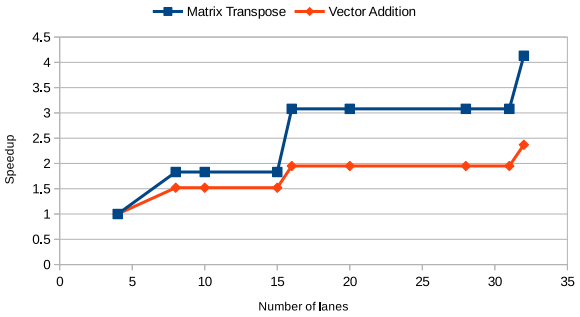


Fig. 15. Scaling the number of lanes.

example, one divergent control flow decision executed by a branch may be negligible if it contains very few lines of code; but it may have a tremendous impact if the warp needs to execute many different code paths with thousands of instructions.

Figure 14(b) shows the control flow efficiency (the ratio of active threads that were not predicated, over the maximum number of threads per warp for each executed SIMD instruction) per-SM. It provides us with the efficiency when using the available execution units. This metric is a good measure that can identify the impact of divergence on overall performance of a benchmark. In Figure 14(b), BS has the lowest control flow efficiency in each SM, while less than 40% of instructions are executed with full utilization of execution units, which means that each divergence path in the kernel contains a significant number of instructions. SP and MT have a similar branch efficiency, but MT has a higher control flow efficiency (97%), while SP is only slight lower (91%) . Clearly, each divergence in SP has a more serious impact on performance than seen in MT.

D. Architecture Exploration

The architectural simulator implemented in the Multi2Sim Kepler simulator provides researchers a flexible simulation framework that can evaluate a wide diversity of design points. To further illustrate the value of Multi2Sim, we present a case study to demonstrate the flexibility of the simulator, where performance significantly varies based on different hardware configurations. Performance is measured using the number of instructions per cycle (IPC).

Figure 15 shows performance achieved by modifying the number of lanes for each private SPU per SM. Our baseline model has 4 lanes in each SPU, and the other performance results are normalized to the baseline performance. In the two benchmarks evaluated, we observe an increase in performance when the number of lanes is a multiple of the warp size (32). This behavior is due to the number of lanes in each private SPU, which determines the number of sub-warps that the private SPU deals with for each warp. When an increase in the number of lanes causes a decrease in the number of sub-warps (e.g., 7 to 8, 15 to 16 and 31 to 32), performance improves. When the number of lanes in each private SPU matches the number of threads in a warp, bottlenecks due to stream serialized core utilization disappear. Compared to matrix transpose, vector addition is more memory intensive. We can see in Figure 13 that matrix transpose has about 5% memory instructions, while vector addition has 18%. This is why when we expand to 16 lanes, we see an extra performance boost in matrix transpose.

VI. Related Work

A number of CPU simulators [12] [13] [14] have been developed to provide effective simulation tools for the computer architecture community. However there are still only a small number of mature GPU architectural simulators available. There continues to be a growing need for architectural GPU simulators that can model a GPU at the ISA (e.g., SASS) level, since a growing number of applications are now leveraging GPUs.

Several studies have attempted to analyzed microarchitecture features through microbenchmarking. Wong et al. [15] analyzed architectural characteristics of the NVIDIA GT200 GPU. Mei et al. [16] attempted to reverse engineer memory hierarchy details for three generations of NVIDIA GPUs (Fermi, Kepler, Maxwell). Although these studies provide efficient ways to analyze application behavior and study future GPUs, a GPU simulator is an essential tool for researchers,

and should allow researchers to explore their own ideas in GPU architecture design.

GPGPU-sim [17] is a popular simulator that supports functional and cycle-level timing simulation based on the NVIDIA Fermi architecture. It has provided the GPU research community with important insights. GPGPUSim also supports PTXPlus, which is an extended form of PTX, introduced in GPGPU-Sim 3.x. The developers claim it allows for a near 1 to 1 mapping of most GT200 (i.e., Fermi) SASS instructions to PTXPlus instructions. However, it only supports execution running at the intermediate language level, and does not capture SASS execution. As we discussed earlier, the number of SIMD instructions for PTX and SASS are, in general, quite different for the same kernel.

Our previous work Multi2sim [18] provides a rich simulation framework for heterogeneous computing. It includes models for superscalar, multithreaded, multi-core, and graphics processors. It provides cycle-level simulation for X86 CPU and AMD GPU architectures.

Gem5-gpu [19] is a heterogeneous simulator that models tightly integrated CPU-GPU systems. It builds upon gem5, a modular full-system CPU simulator, and GPGPU-sim. Compared to GPGPU-sim, Gem5-gpu also supports CPU simulation, but again, only supports PTX-level simulation for the GPU.

Barra [20] provides a ISA-level functional simulation for the NVIDIA Tesla GPU architecture. It runs CUDA executables without modification. Since NVIDIA's ISA documentation is not publicly available, similar to our approach, the user needs to reverse engineering the NVIDIA ISA. It can also perform parallel functional simulation using a multi-core processor or SIMD instruction set. Unfortunately, we cannot produce accurate performance models for GPU architectures, lacking support of cycle-level timing simulation.

Ocelot [21] is a widely used dynamic compilation framework, as well as a functional simulator, that executes programs working at the PTX virtual ISA level. Taking NVIDIAs CUDA PTX code as input, we can either emulate or dynamically translate instructions to multiple platforms such as multicore CPUs, NVIDIA GPUs, and AMD GPUs. Working off an existing compiler infrastructure, Ocelot provides facilities for compiler research, including interfaces, to provide an internal representation of the PTX programs in support of optimization passes for massively data parallel computer kernels.

VII. Conclusions and Future Work

In this paper we presented Multi2Sim Kepler, a detailed performance simulator that simulates NVIDIA's shader assembly (SASS) code based on the NVIDIA Kepler architecture. To illustrate the utility of this framework, we provided a number of example architectural studies, exploring the micro-architecture of the SM front-end, SIMD instruction pipeline, and compute resource partitioning. We also discussed features of the the NVIDIA SASS code and highlighted the differences between SASS and PTX through a series of benchmarks.

The framework has been developed through an intensive analysis and reverse engineering of Kepler hardware. We have developed a cycle-accurate architectural simulation at the ISA level. Based on this simulation framework, researchers will have a much clearer look into the NVIDIA's architectural features when they evaluate their designs using our simulator. To the best of our knowledge, this is the first GPU simulator running on NVIDIA Kepler SASS. The source of the simulator is available on the Multi2Sim website.

We plan to continue to improve the accuracy of our simulator, focusing on the memory hierarchy model. In terms of ongoing work with Multi2Sim, we will be providing the ability to replace the computing pipeline of the GPU with a FPGA-like programmable accelerator fabric. This work has been supported in part by NSF awards CNS-1525412 and CNS-1319501.

References

[1] *NVIDIA CUDA C Programming Guide, v8.0*, 2016.

[2] D. R. Kaeli, P. Mistry, D. Schaa, and D. P. Zhang, "Heterogeneous computing with OpenCL 2.0," San Francisco, CA, USA, 2015.

[3] W.-M.Hwu, *GPU Computing Gems Emerald Edition*. Elsevier, 2011.

[4] *CUDA Runtime API, v8.0*, 2016.

[5] *CUDA Driver API, v8.0*, 2016.

[6] *NVIDIA Kepler GK110 Architecture Whitepaper*, 2012.

[7] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "Nvidia tesla: A unified graphics and computing architecture," *IEEE micro*, vol. 28, no. 2, pp. 39–55, 2008.

[8] *NVIDIA Fermi Compute Architecture Whitepaper*, 2009.

[9] *NVIDIA Geforce GTX 980 Whitepaper*, 2014.

[10] *NVIDIA PTX:Parallel Thread Execution ISA, v5.0*, 2016.

[11] *NVIDIA Tesla K20X GPU Accelerator*, 2013.

[12] D. Burger and T. M. Austin, "The simplescalar tool set, version 2.0," *ACM SIGARCH Comp. Arch. News*, vol. 25, no. 3, pp. 13–25, 1997.

[13] Y. Lee, A. Waterman, R. Avizienis, H. Cook, C. Sun, V. Stojanović, and K. Asanović, "A 45nm 1.3 ghz 16.7 double-precision gflops/w risc-v processor with vector accelerators," in *40th European Solid State Circuits Conference (ESSCIRC)*. IEEE, 2014, pp. 199–202.

[14] N. Binkert, B. Beckmann, G. Black, S. K. Reinhardt, A. Saidi, A. Basu, J. Hestness, D. R. Hower, T. Krishna, S. Sardashti *et al.*, "The gem5 simulator," *ACM SIGARCH Comp. Arch. News*, vol. 39, no. 2, pp. 1–7, 2011.

[15] H. Wong, M.-M. Papadopoulou, M. Sadooghi-Alvandi, and A. Moshovos, "Demystifying gpu microarchitecture through microbenchmarking," in *IEEE Int. Symp. on Performance Analysis of Systems & Software*, 2010, pp. 235–246.

[16] X. Mei and X. Chu, "Dissecting gpu memory hierarchy through microbenchmarking," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 1, pp. 72–86, 2017.

[17] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *IEEE Int. Symp. on Performance Analysis of Systems and Software*, 2009, pp. 163–174.

[18] R. Ubal, B. Jang, P. Mistry, D. Schaa, and D. Kaeli, " Multi2Sim: A Simulation Framework for CPU-GPU Computing ," in *Proc. of the 21st International Conference on Parallel Architectures and Compilation Techniques*, Sep. 2012.

[19] J. Power, J. Hestness, M. S. Orr, M. D. Hill, and D. A. Wood, "gem5-gpu: A heterogeneous cpu-gpu simulator," *IEEE Computer Architecture Letters*, vol. 14, no. 1, pp. 34–36, 2015.

[20] S. Collange, M. Daumas, D. Defour, and D. Parello, "Barra: A parallel functional simulator for gpgpu," in *2010 IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*, 2010, pp. 351–360.

[21] G. F. Diamos, A. R. Kerr, S. Yalamanchili, and N. Clark, "Ocelot: a dynamic optimization framework for bulk-synchronous applications in heterogeneous systems," in *ACM Proc. of the 19th Int. Conf. on Parallel Architectures and Compilation Techniques*, 2010, pp. 353–364.